LÝ THUYẾT ĐỒ THỊ

knguyen – vhung – vanh

Đồ thị là gì?

Đồ thị là cấu trúc toán học gồm:

$$G = (V, E)$$

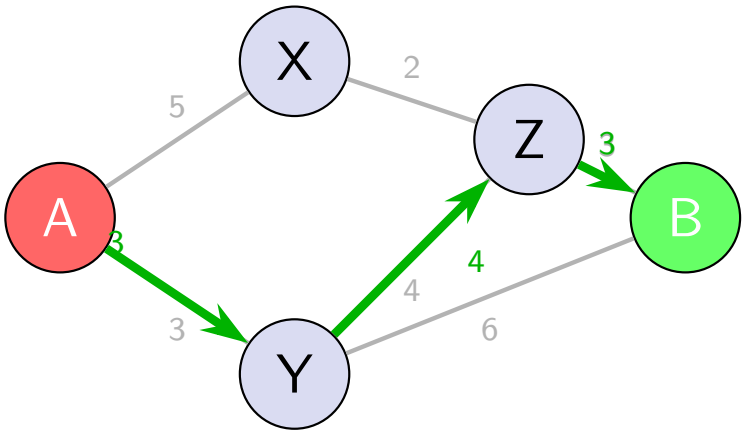
- V : tập các đỉnh
- E : tập các cạnh nối các đỉnh

Google Maps: Tìm đường đi ngắn nhất

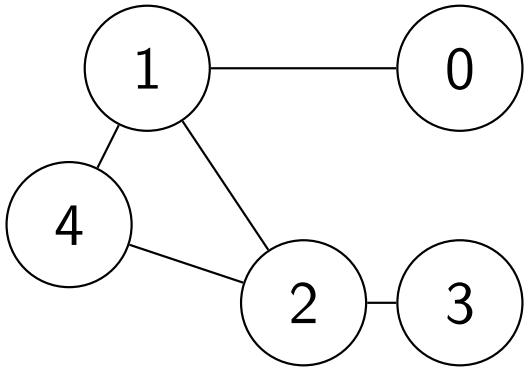
Bài toán: Tìm đường đi ngắn nhất từ **Điểm A** đến **Điểm B**.

- Các giao lộ → Đỉnh (Vertices)
- Các con đường → Cạnh (Edges)
- Thời gian/Khoảng cách → Trọng số (Weights)

Thuật toán Dijkstra thường được sử dụng cho bài toán này.



Ví dụ đồ thị

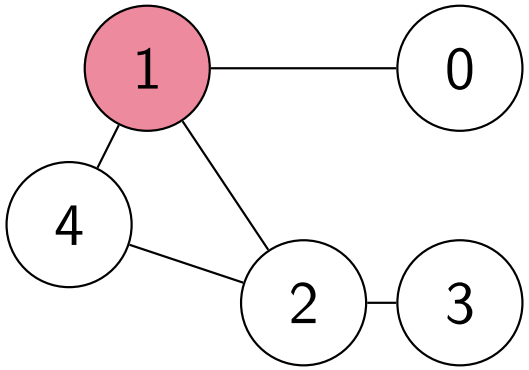


Thành phần của đồ thị

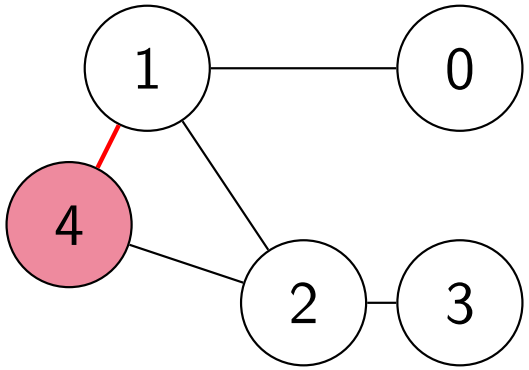
- Đỉnh (Vertices)
- Cạnh (Edges)
- Bậc của đỉnh
- Đường đi (Path)

VÍ DỤ VỀ ĐƯỜNG ĐI CỦA MỘT ĐỒ THỊ

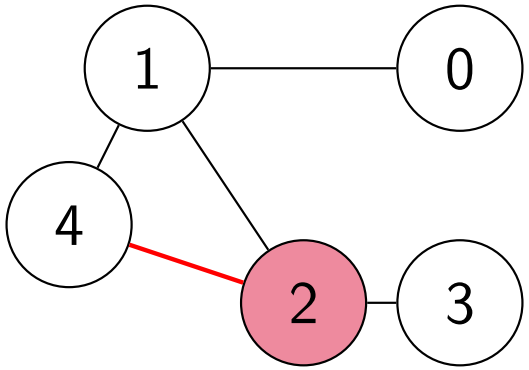
Bước 1: Bắt đầu



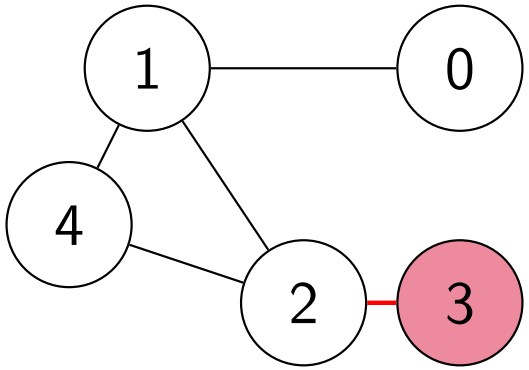
Bước 2: Di chuyển đến đỉnh 4



Bước 3: Di chuyển đến đỉnh 2



Bước 4: Di chuyển đến đỉnh 3



Biểu diễn: Danh sách kề (Adjacency List)

Cấu trúc: Mỗi đỉnh lưu một danh sách các đỉnh láng giềng.

- **Đỉnh 1:** [4, 0, 2]
- **Đỉnh 2:** [1, 3, 4]
- **Đỉnh 3:** [2]

Ưu điểm:

- **Tiết kiệm bộ nhớ:** Chỉ tốn $O(V + E)$.
- **Duyệt nhanh:** Duyệt các đỉnh kề mất $O(deg(u))$.
- Phù hợp cho đồ thị lớn, đồ thị thưa.

Biểu diễn: Danh sách kề

Danh sách kề sử dụng mảng vector để lưu trữ các đỉnh láng giềng.

```
int main() {
    int n, m; // số đỉnh, số cạnh
    cin >> n >> m;
    for(int i=0; i<m; i++){
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u); // Đồ thị vô hướng
    }
    return 0;
}
```

Biểu diễn: Danh sách cạnh (Edge List)

- Lưu trữ danh sách các cặp đỉnh (u, v) tạo thành cạnh.
- Rất hiệu quả khi thuật toán chỉ cần duyệt qua từng cạnh một.
  Đặc điểm:
  - Dễ cài đặt, tốn ít bộ nhớ.
  - Phù hợp với các thuật toán chỉ quan tâm đến các cạnh (như Kruskal, Bellman-Ford).
  - Không hiệu quả khi cần tìm các đỉnh kề của một đỉnh cụ thể.

Ví dụ:

STT	Đỉnh u	Đỉnh v
1	1	4
2	1	0
3	1	2
4	2	3
5	4	2

Biểu diễn: Danh sách cạnh

Danh sách cạnh lưu trữ đồ thị dưới dạng tập hợp các cặp đỉnh.

```
struct Edge {
    int u, v;
};
vector<Edge> edgeList;

// Thêm cạnh
edgeList.push_back({u, v});
```

Biểu diễn: Ma trận kề (Adjacency Matrix)

Ma trận A kích thước $N \times N$:

- $A[i][j] = 1$: Nếu có cạnh nối i và j.
- $A[i][j] = 0$: Nếu không có.

Đồ thị 3 đỉnh:

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

(Dòng 1 nối với đỉnh 2, Dòng 2 nối 1 và 3, v.v.)

Nhược điểm: Tốn bộ nhớ $O(N^2)$, không tối ưu với đồ thị có hàng chục ngàn đỉnh.

Biểu diễn: Ma trận kề

```
// Ma trận kích thước tối đa 100x100
int adj[100][100];
int main() {
    int n, m; // số đỉnh, số cạnh
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        // Đồ thị vô hướng: đánh dấu 1 ở cả 2 chiều
        adj[u][v] = 1;
        adj[v][u] = 1;
    }
    return 0;
}
```

Thuật toán DFS (Depth-First Search)

- **Nguyên lý:** "Đi sâu nhất có thể" trước khi quay lui (Backtracking).
- **Cấu trúc dữ liệu:** Sử dụng **Ngăn xếp (Stack)** hoặc **Đệ quy** (chính là Stack của hệ thống).
- **Các bước:**
  1. Thăm đỉnh u, đánh dấu u là đã thăm (visited).
  2. Với mỗi đỉnh v kề với u:
     - Nếu v chưa được thăm, thực hiện gọi đệ quy `DFS(v)`.

Lưu ý: DFS rất dễ cài đặt nhờ đệ quy, nhưng cần cẩn thận với đồ thị quá lớn vì có thể gây tràn Stack (Stack Overflow).

Cài đặt DFS với C++ (Đệ quy)

```
vector<int> adj[100]; // Danh sách kề
bool visited[100];    // Mảng đánh dấu đã thăm

void DFS(int u) {
    visited[u] = true;      // Đánh dấu u đã thăm
    // Duyệt qua các đỉnh kề v
    for(int v : adj[u]) {
        if(!visited[v]) {
            DFS(v);         // Gọi đệ quy cho v
        }
    }
}
```

Ghi chú: visited[] đóng vai trò cực kỳ quan trọng để ngăn chương trình lặp vô tận (nếu đồ thị có chu trình).

Thuật toán BFS (Breadth-First Search)

- **Nguyên lý:** Duyệt theo từng lớp, bắt đầu từ đỉnh nguồn, lan rộng ra các đỉnh láng giềng.
- **Cấu trúc dữ liệu:** Sử dụng **Hàng đợi (Queue)** theo cơ chế FIFO (Vào trước - Ra trước).
- **Các bước:**
  1. Đưa đỉnh nguồn vào Queue và đánh dấu đã thăm.
  2. Lặp khi Queue không rỗng:
     - Lấy đỉnh u ra khỏi Queue.
     - Xét các đỉnh v kề với u: Nếu chưa thăm, đưa v vào Queue và đánh dấu.

Cài đặt BFS với C++

```
void BFS(int start) {
    queue<int> q;
    q.push(start);
    visited[start] = true;
    while(!q.empty()){
        int u = q.front(); q.pop();
        cout << u << " ";
        for(int v : adj[u]){
            if(!visited[v]){
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

Ứng dụng của đồ thị

- Google Maps (Tìm đường đi ngắn nhất)
- Mạng xã hội (Mối quan hệ)
- Internet (Liên kết trang web)
- Game AI (Di chuyển nhân vật)

CẢM ƠN